INTERNSPARK • TASK 3

Weather Explorer
API Integration Project

Student name	Sahasra Preetam
Task	Task 3 — API Integration Project-Weather Explorer
Technologies	HTML5, CSS3, Bootstrap 5, JavaScript
Project folder	InternsparkTask-3-API-Integration

1. Project Overview

Weather Explorer is a responsive weather application built with HTML, CSS, JavaScript, and Bootstrap. A user enters a city name; the application looks up the city's geographic coordinates, requests current weather data for those coordinates, and displays the returned information.

2. Deliverables

A. Hosted Live Project

Live URL: https://sahapreeth23-rock-it.github.io/InternSpark-Task-3-API-Integration/

Deployment status: Successful

B. GitHub Repository

Repository URL: https://github.com/sahapreeth23-rock-it/InternSpark-Task-3-API-Integration

Project files:

- index.html — page structure and weather result elements
- style.css — visual styling and responsive adjustments
- script.js — city search, API requests, error handling, and result display

C. Short Write-up on the API Used

This project uses the Open-Meteo Geocoding API and Forecast API. The geocoding endpoint searches for the city entered by the user and returns location details, including latitude and longitude. The application then sends those coordinates to the Forecast API to request current weather fields.

Geocoding endpoint pattern:

```
https://geocoding-api.open-meteo.com/v1/search
```

Forecast endpoint pattern:

```
https://api.open-meteo.com/v1/forecast
```

The current-weather request asks for temperature, relative humidity, apparent temperature, day/night indicator, precipitation, weather code, and wind speed, with the location's time zone. JavaScript's fetch() method sends the HTTP requests, and response.json() parses the returned JSON. The weather code is mapped to a short description and an emoji icon for display.

API key: No API key is required for standard non-commercial use. Check Open-Meteo's current terms and attribution requirements before public or commercial use.

Official documentation: https://open-meteo.com/en/docs

Geocoding documentation: https://open-meteo.com/en/docs/geocoding-api

3. Technologies Used

- HTML5 — page structure and input form
- CSS3 — styling and responsive design
- Bootstrap 5 — responsive layout and interface components
- JavaScript — form handling, asynchronous fetch requests, JSON parsing, and error handling
- Open-Meteo APIs — city geocoding and weather data
- Git and GitHub — version control and source-code hosting
- GitHub Pages — static website hosting (after deployment)

4. Main Features

- Search for weather by city name.
- Automatically look up city coordinates using the geocoding API.
- Display temperature, humidity, wind speed, weather description, and icon.
- Show a loading indicator while requests are processed.
- Display an error when a city is not found or a request fails.
- Responsive interface intended for desktop and mobile screens.

5. How the Application Works

1. The user enters a city name and submits the form.
2. JavaScript sends a request to the Geocoding API.
3. The first matching location's latitude and longitude are read from the response.
4. JavaScript requests current weather data for those coordinates from the Forecast API.
5. The JSON response is parsed and relevant weather fields are displayed.
6. If a request fails or the city is not found, an error message is displayed.

6. Code Snippets

Requesting a city match:

```
const geoResponse = await fetch(geoURL);
const geoData = await geoResponse.json();
```

Requesting and parsing weather data:

```
const weatherResponse = await fetch(weatherURL);
const weatherData = await weatherResponse.json();
```

7. Testing Checklist

☐Search for a valid city (for example, Hyderabad).

☐Search for a city in another country (for example, London).

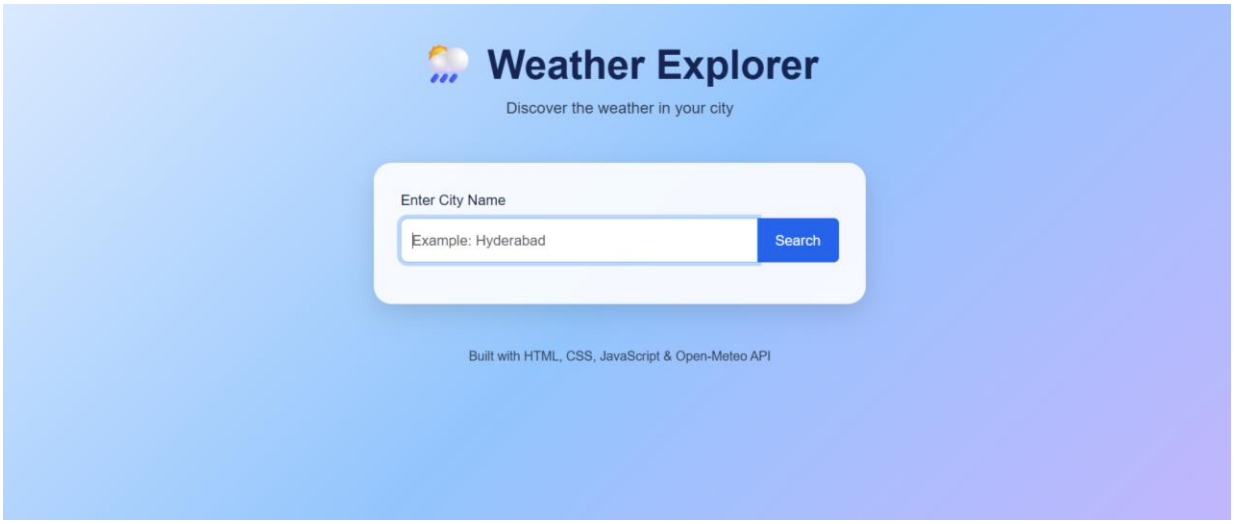
☐Enter an invalid city name and verify a helpful error appears.

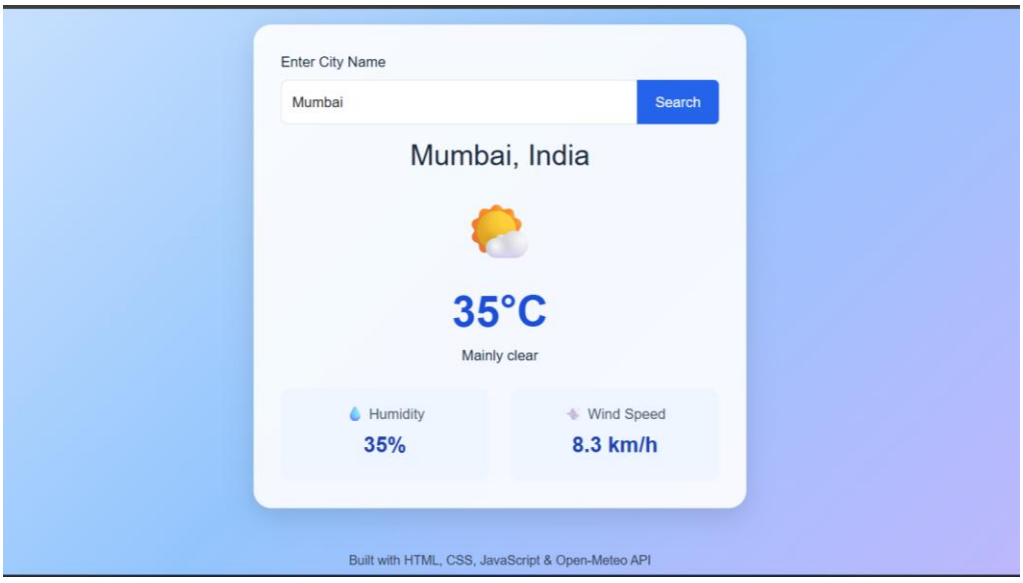
☐Submit an empty search field and verify validation.

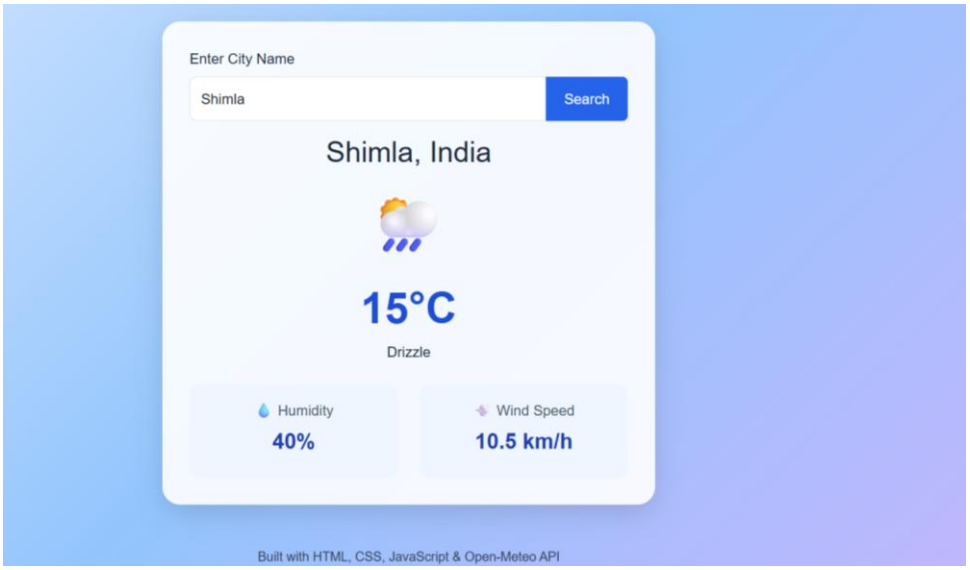
☐Check the layout at desktop and mobile widths.

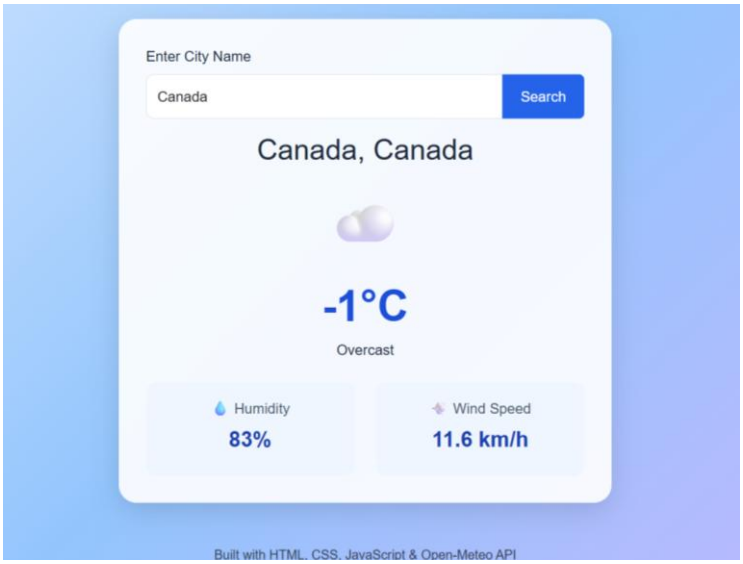
☐Open the deployed public URL and confirm the page and API requests work.

8. Project Screenshots









INTERNSPARK • TASK 3

Weather Explorer
API Integration Project

Student name	Sahasra Preetam
Task	Task 3 — API Integration Project-Weather Explorer
Technologies	HTML5, CSS3, Bootstrap 5, JavaScript
Project folder	InternsparkTask-3-API-Integration

1. Project Overview

Weather Explorer is a responsive weather application built with HTML, CSS, JavaScript, and Bootstrap. A user enters a city name; the application looks up the city's geographic coordinates, requests current weather data for those coordinates, and displays the returned information.

2. Deliverables

A. Hosted Live Project

Live URL: https://sahapreeth23-rock-it.github.io/InternSpark-Task-3-API-Integration/

Deployment status: Successful

B. GitHub Repository

Repository URL: https://github.com/sahapreeth23-rock-it/InternSpark-Task-3-API-Integration

Project files:

- index.html — page structure and weather result elements
- style.css — visual styling and responsive adjustments
- script.js — city search, API requests, error handling, and result display

C. Short Write-up on the API Used

This project uses the Open-Meteo Geocoding API and Forecast API. The geocoding endpoint searches for the city entered by the user and returns location details, including latitude and longitude. The application then sends those coordinates to the Forecast API to request current weather fields.

Geocoding endpoint pattern:

```
https://geocoding-api.open-meteo.com/v1/search
```

Forecast endpoint pattern:

```
https://api.open-meteo.com/v1/forecast
```

The current-weather request asks for temperature, relative humidity, apparent temperature, day/night indicator, precipitation, weather code, and wind speed, with the location's time zone. JavaScript's fetch() method sends the HTTP requests, and response.json() parses the returned JSON. The weather code is mapped to a short description and an emoji icon for display.

API key: No API key is required for standard non-commercial use. Check Open-Meteo's current terms and attribution requirements before public or commercial use.

Official documentation: https://open-meteo.com/en/docs

Geocoding documentation: https://open-meteo.com/en/docs/geocoding-api

3. Technologies Used

- HTML5 — page structure and input form
- CSS3 — styling and responsive design
- Bootstrap 5 — responsive layout and interface components
- JavaScript — form handling, asynchronous fetch requests, JSON parsing, and error handling
- Open-Meteo APIs — city geocoding and weather data
- Git and GitHub — version control and source-code hosting
- GitHub Pages — static website hosting (after deployment)

4. Main Features

- Search for weather by city name.
- Automatically look up city coordinates using the geocoding API.
- Display temperature, humidity, wind speed, weather description, and icon.
- Show a loading indicator while requests are processed.
- Display an error when a city is not found or a request fails.
- Responsive interface intended for desktop and mobile screens.

5. How the Application Works

7. The user enters a city name and submits the form.
8. JavaScript sends a request to the Geocoding API.
9. The first matching location's latitude and longitude are read from the response.
10. JavaScript requests current weather data for those coordinates from the Forecast API.
11. The JSON response is parsed and relevant weather fields are displayed.
12. If a request fails or the city is not found, an error message is displayed.

6. Code Snippets

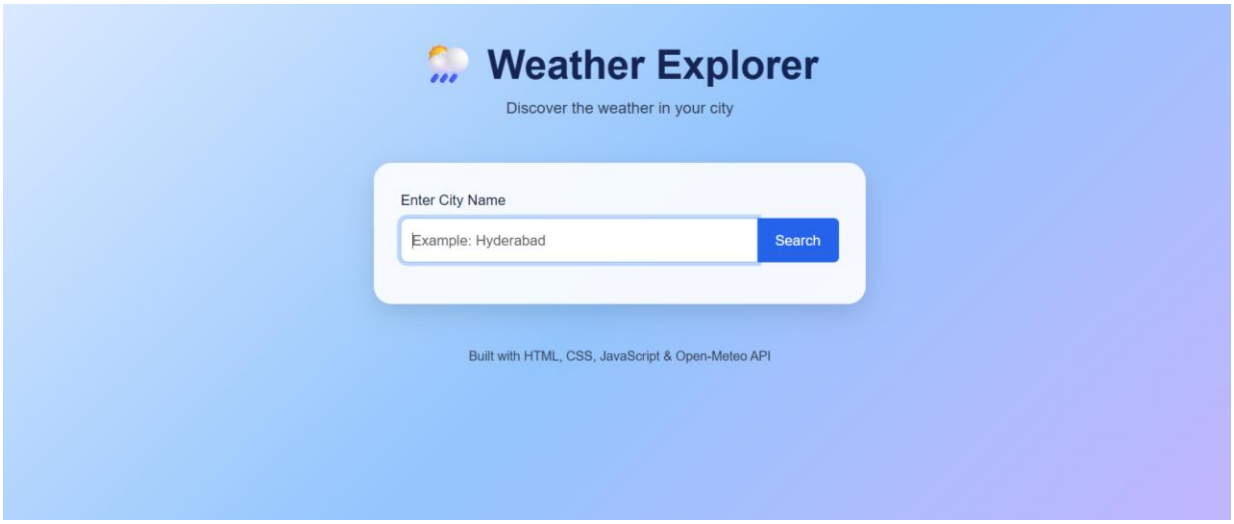
Requesting a city match:

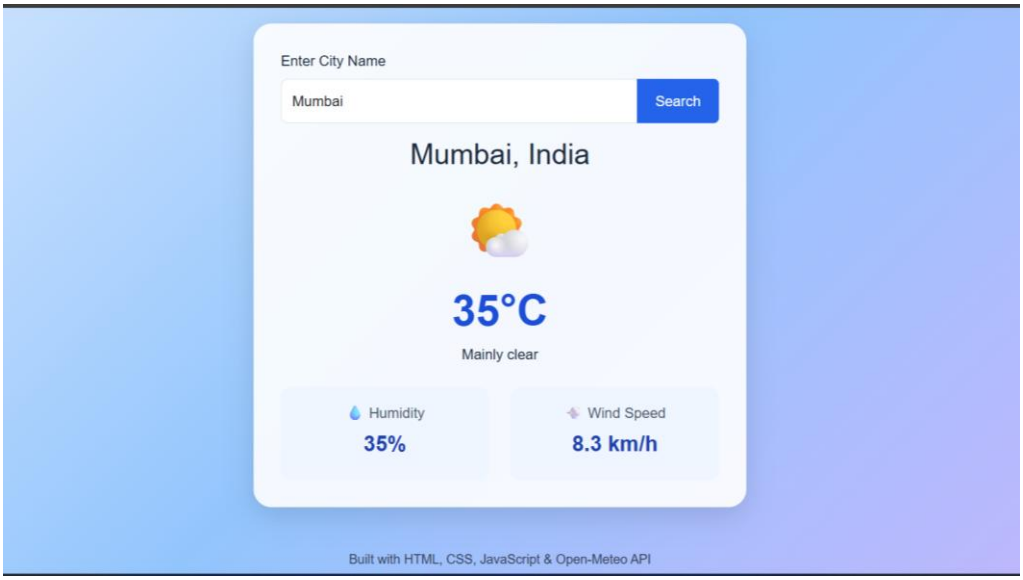
```
const geoResponse = await fetch(geoURL);
const geoData = await geoResponse.json();
```

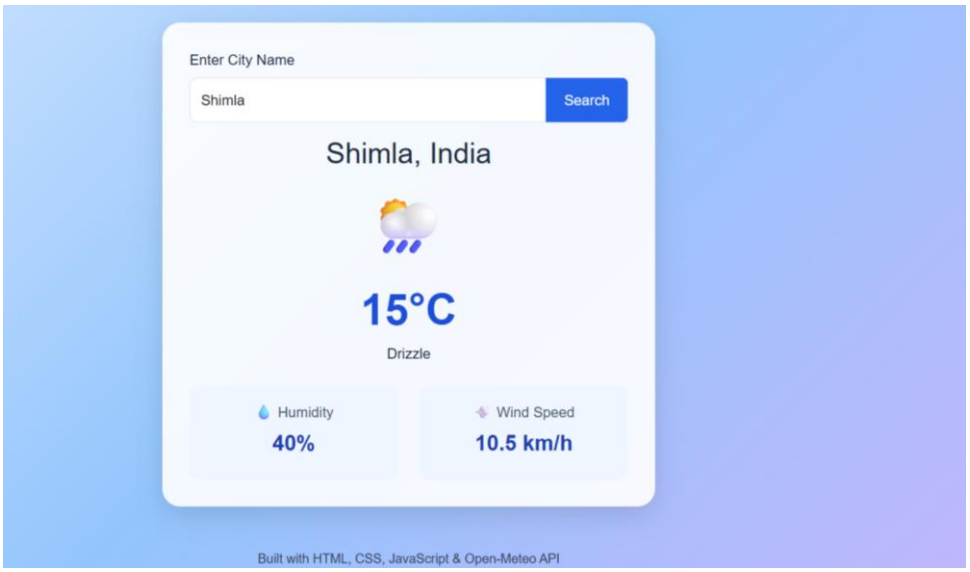
Requesting and parsing weather data:

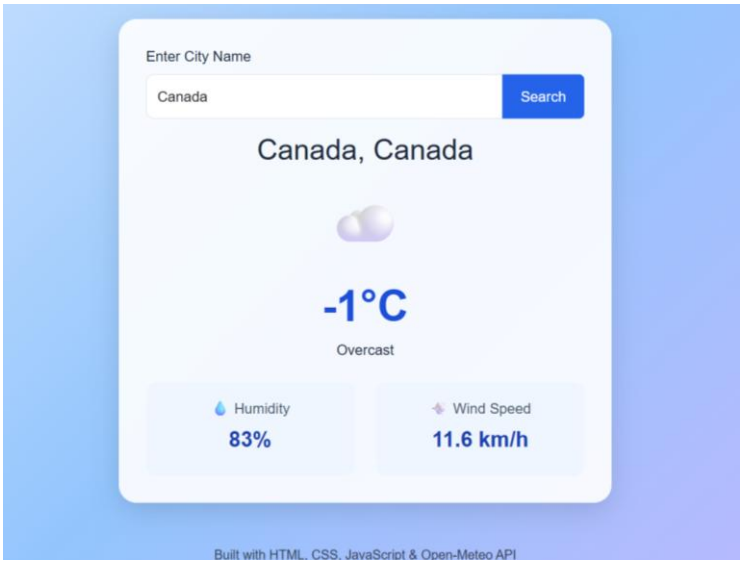
```
const weatherResponse = await fetch(weatherURL);
const weatherData = await weatherResponse.json();
```

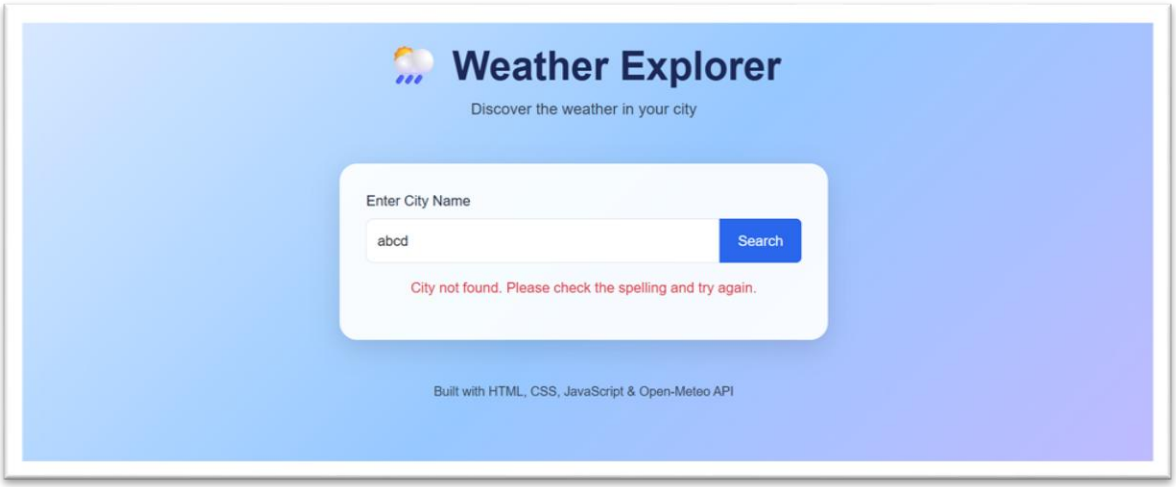
7. Project Screenshots

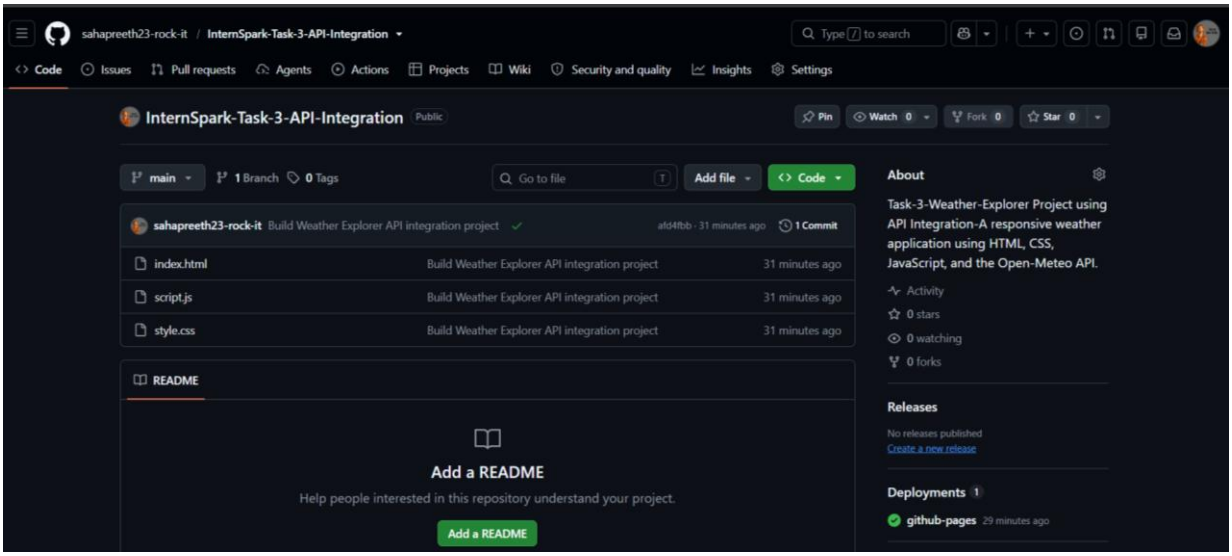



Enter City Name
Mumbai
Search
Mumbai, India
35°C
Mainly clear
Humidity
35%
Wind Speed
8.3 km/h
Built with HTML, CSS, JavaScript & Open-Meteo API


Enter City Name
Shimla
Search
Shimla, India
15°C
Drizzle
Humidity
40%
Wind Speed
10.5 km/h
Built with HTML, CSS, JavaScript & Open-Meteo API


Enter City Name
Canada
Search
Canada, Canada
-1°C
Overcast
Humidity
83%
Wind Speed
11.6 km/h
Built with HTML, CSS, JavaScript & Open-Meteo API





8. Conclusion

This project demonstrates a basic API integration workflow in a frontend application: collecting user input, requesting data from external endpoints, processing JSON responses, handling errors, and updating the interface dynamically. Add the actual repository and hosted URLs above after publication.

End of Task 3 Submission Report